Unit Testing using JUnit 5 & Mockito

Complete Interview Preparation Roadmap for Java + Spring Boot Developers

> **Overview:** This roadmap covers 95% of the interview questions asked on JUnit and Mockito for Java + Spring Boot roles at top IT companies. Master these concepts from beginner to advanced.

1. What is Unit Testing?

Definition: Unit Testing is the process of testing one small unit of code independently.

A unit can be a Method, Function, Service Class, or Utility Class.

Example:

```
public int add(int a, int b){
    return a + b;
}
```

Instead of running the whole application, we test only this method.

Why Unit Testing?

Without Testing	With Unit Testing
Developer writes code ↓ Runs entire application ↓ Checks manually ↓ *Time consuming*	Developer writes code ↓ Runs JUnit Test ↓ JUnit automatically checks result ↓ *Fast*

Benefits:

- Finds bugs early
- Prevents future bugs
- Improves code quality
- Safe refactoring
- Faster development

2. Types of Testing

Testing Type	Purpose
Unit Testing	Tests one class or method (e.g., Test only Service)
Integration Testing	Tests communication between classes/modules (e.g., Service + Repository + Database)
System Testing	Tests the entire application
Acceptance Testing	Tests business requirements

3. What is JUnit?

JUnit is the most popular Java testing framework. It is used to write test cases, execute test cases, compare expected and actual results, and generate test reports.

Dependency (Maven):

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
```

This includes: JUnit 5, Mockito, AssertJ, and Hamcrest.

4. JUnit Lifecycle

The standard execution flow of JUnit 5 annotations:

@BeforeAll → @BeforeEach → @Test → @AfterEach → @AfterAll

@BeforeAll

Runs **only once** before all test methods. Useful for creating a Database Connection.

```
@BeforeAll
static void setupDatabase() { }
```

@BeforeEach

Runs **before every test**. Useful to create fresh objects (like Employee) for each test.

```
@BeforeEach
void init() { }
```

@Test

Marks a test method. Without `@Test`, JUnit ignores the method.

```
@Test
void testSave() { }
```

@AfterEach: Runs after every test. Used for cleanup.

@AfterAll: Runs once after all tests. Used to close resources.

5. Assertions

Assertions compare **Expected** vs **Actual** values.

- `assertEquals(10, result);` — Pass if Expected = 10, Actual = 10
- `assertNotEquals(20, result);`
- `assertTrue(age > 18);`
- `assertFalse(list.isEmpty());`
- `assertNull(employee);`
- `assertNotNull(employee);`
- `assertThrows(RuntimeException.class, () -> service.getEmployee(1L));` — Tests exception.
- `assertDoesNotThrow(() -> service.save(employee));`

6. Mockito Introduction

Mockito creates fake objects. Should a unit test connect to MySQL? **No.** Mockito replaces the Repository.

`EmployeeService` → `Fake Repository` → `No Database`

7. Mock Object

- **Real Object:** Repository → Database
- **Mock Object:** Repository → Returns Fake Data

8. Key Annotations

@Mock

Creates fake dependency (Fake Repository).

```
@Mock
EmployeeRepository repository;
```

@InjectMocks

Creates the actual class and injects the mocked dependencies into it.

```
@InjectMocks
EmployeeService service;
```

@ExtendWith(MockitoExtension.class)

This is the preferred approach in JUnit 5 to initialize mocks.

```
@ExtendWith(MockitoExtension.class)
class EmployeeServiceTest {
    @Mock
    EmployeeRepository repository;

    @InjectMocks
    EmployeeService service;
}
```

9. Mocking Behavior

when() and thenReturn()

Defines how the mock should behave and what fake object it returns.

```
when(repository.findById(1L)).thenReturn(Optional.of(employee));
when(repository.save(employee)).thenReturn(employee);
```

thenThrow()

Simulates an exception thrown by the mocked dependency.

```
when(repository.findById(1L)).thenThrow(new RuntimeException());
```

10. Verification (verify)

Checks method invocation. Was a specific method called on the mock?

- `verify(repository).save(employee);` — Was save() called?
- `verify(repository, times(2)).save(employee);` — Called exactly 2 times.
- `verify(repository, never()).delete(employee);` — Ensures delete was never called.
- `verify(repository, atLeast(1)).save(employee);`
- `verify(repository, atMost(2)).save(employee);`

11. Argument Matchers

- `any()`: Matches any object of a given type.
  `when(repository.save(any(Employee.class)))`
- `eq()`: Exact value matcher.
  `verify(repository).findById(eq(1L));`

12. ArgumentCaptor

Captures arguments passed to a mocked method, useful for checking object values before saving.

```
ArgumentCaptor<Employee> captor = ArgumentCaptor.forClass(Employee.class);
verify(repository).save(captor.capture());
Employee emp = captor.getValue();
```

13. Mock vs Spy

Spy = Partial Mock. Real methods execute unless explicitly mocked.

```
List<String> list = new ArrayList<>();
List<String> spyList = spy(list);
doReturn("Hello").when(spyList).get(0);
doThrow(new RuntimeException()).when(service).delete();
```

Mock	Spy
Completely fake	Partially fake
No real methods run	Real methods run
Faster	Used for partial mocking

14. Complete Spring Boot Example

Repository & Service

```
public interface EmployeeRepository extends JpaRepository<Employee,Long> { }

@Service
public class EmployeeService {
    @Autowired
    EmployeeRepository repository;

    public Employee getEmployee(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new RuntimeException("Not Found"));
    }
}
```

Test Class

```
@ExtendWith(MockitoExtension.class)
class EmployeeServiceTest {

    @Mock
    EmployeeRepository repository;

    @InjectMocks
    EmployeeService service;

    @Test
    void testGetEmployee() {
        Employee emp = new Employee();
        emp.setId(1L);
        emp.setName("John");

        // 1. Mock Behavior
        when(repository.findById(1L)).thenReturn(Optional.of(emp));

        // 2. Call Service
        Employee result = service.getEmployee(1L);

        // 3. Assert Result
        assertEquals("John", result.getName());

        // 4. Verify Invocation
        verify(repository).findById(1L);
    }
}
```

15. Best Practices

✅ Test one method at a time.

✅ One test should verify one behavior.

✅ Use meaningful test method names (`testGetEmployeeById()` instead of `test1()`).

✅ Never connect to a real database in unit tests.

✅ Mock external services (REST APIs, databases, Kafka, etc.).

16. Common Interview Questions

Q1. What is Unit Testing?

Testing an individual method or class in isolation to ensure it behaves as expected.

Q2. Difference between JUnit and Mockito?

JUnit:

Testing framework that executes tests and provides assertions.

Mockito:

Mocking framework that creates mock objects and stubs/verifies method calls.

Q3. What is Mocking?

Replacing real dependencies with fake objects to isolate the class under test.

Q4. Why use Mockito?

To avoid calling real databases, REST APIs, file systems, or other external dependencies during unit tests.

Q5. Difference between @Mock and @InjectMocks?

`@Mock` creates a fake dependency. `@InjectMocks` creates the real class under test and injects the mocked dependencies into it.

Q6. Why use verify()?

To ensure that expected interactions with dependencies actually occurred (e.g., checking that `repository.save(employee)` was called exactly once).

Q7. Difference between when().thenReturn() and verify()?

`when().thenReturn()` defines *how* a mock should behave. `verify()` checks *whether* a method on the mock was called.

Q8. Why don't we use a real database in unit testing?

Because unit tests should be Fast, Independent, Repeatable, and Focused only on the business logic of the class being tested. Using a real database is more appropriate for integration testing.

17. Complete Interview Summary Cheat Sheet

Concept	Purpose
Unit Testing	Tests a single class or method in isolation
JUnit	Framework to write and run tests
Mockito	Framework to create mock objects
@Test	Marks a test method
@BeforeEach / @AfterEach	Setup and cleanup before/after each test
@BeforeAll / @AfterAll	One-time setup and cleanup for all tests
Assertions	Validate expected vs. actual results
@Mock	Creates a fake dependency
@InjectMocks	Creates the class under test and injects mocks
@ExtendWith(...)	Initializes Mockito annotations in JUnit 5
when().thenReturn()	Defines mock behavior
thenThrow()	Simulates exceptions

verify()	Confirms method invocations
times(), never(), etc.	Verifies invocation counts
any(), eq()	Argument matchers
ArgumentCaptor	Captures and inspects method arguments
spy()	Creates a partial mock of a real object
doReturn(), doThrow()	Stubbing methods on spies or void methods

> **If you understand these concepts and can explain the Spring Boot service-layer example confidently, you'll be well prepared for most JUnit and Mockito interview questions for Java/ Spring Boot roles. Good Luck!**